

User Activity Mouse Dynamics - Practical 5 Report

Introduction

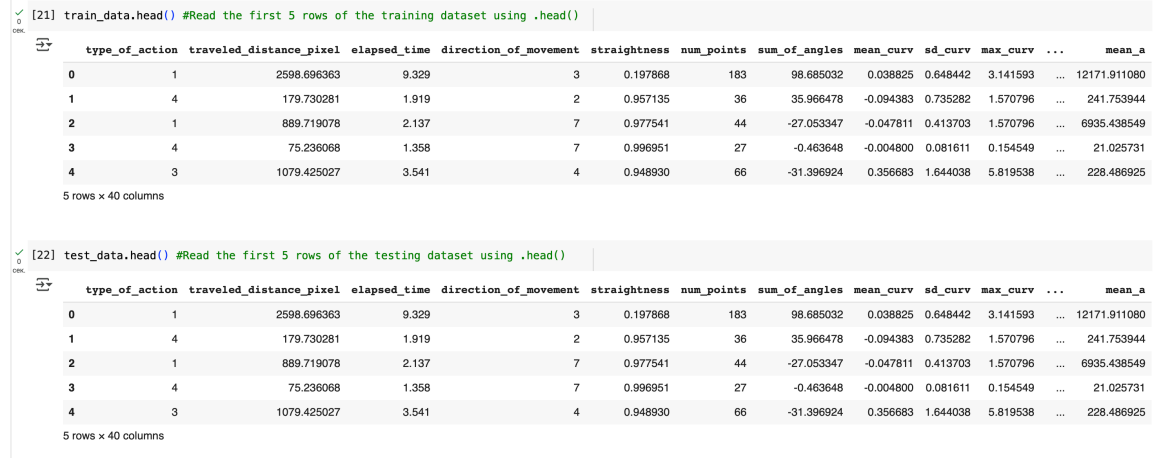
This report presents the implementation of Practical 5: User Activity Mouse Dynamics Analysis. The main objective is to authenticate users based on their mouse dynamics using machine learning techniques.

Step 1: Load the Dataset

The datasets for both training and testing were loaded using pandas:

```
train_data = pd.read_csv(path_to_train)
```

```
test_data = pd.read_csv(path_to_test)
```



The screenshot displays two Jupyter Notebook cells. Cell [21] shows the command `train_data.head()` and its output, which is the first 5 rows of the training dataset. Cell [22] shows the command `test_data.head()` and its output, which is the first 5 rows of the testing dataset. Both datasets have 40 columns and 5 rows are displayed.

	type_of_action	traveled_distance_pixel	elapsed_time	direction_of_movement	straightness	num_points	sum_of_angles	mean_curv	sd_curv	max_curv	...	mean_a
0	1	2598.696363	9.329	3	0.197868	183	98.685032	0.038825	0.648442	3.141593	...	12171.911080
1	4	179.730281	1.919	2	0.957135	36	35.966478	-0.094383	0.735282	1.570796	...	241.753944
2	1	889.719078	2.137	7	0.977541	44	-27.053347	-0.047811	0.413703	1.570796	...	6935.438549
3	4	75.236068	1.358	7	0.996951	27	-0.463648	-0.004800	0.081611	0.154549	...	21.025731
4	3	1079.425027	3.541	4	0.948930	66	-31.396924	0.356683	1.644038	5.819538	...	228.486925

5 rows x 40 columns

	type_of_action	traveled_distance_pixel	elapsed_time	direction_of_movement	straightness	num_points	sum_of_angles	mean_curv	sd_curv	max_curv	...	mean_a
0	1	2598.696363	9.329	3	0.197868	183	98.685032	0.038825	0.648442	3.141593	...	12171.911080
1	4	179.730281	1.919	2	0.957135	36	35.966478	-0.094383	0.735282	1.570796	...	241.753944
2	1	889.719078	2.137	7	0.977541	44	-27.053347	-0.047811	0.413703	1.570796	...	6935.438549
3	4	75.236068	1.358	7	0.996951	27	-0.463648	-0.004800	0.081611	0.154549	...	21.025731
4	3	1079.425027	3.541	4	0.948930	66	-31.396924	0.356683	1.644038	5.819538	...	228.486925

5 rows x 40 columns

Figure 1: "Screenshot: Loading and previewing training/test data"

Step 2: Explore the Dataset

First few rows and basic statistics were explored using `.head()` and `.describe()` functions. The dataset contains 40 features including mouse movement, click, speed, and acceleration metrics.

```
train_data.info() #display the columns information (data type, names and Non-null count) in the training dataset, accomplished using .info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 40 columns):
 #   Column                                Non-Null Count  Dtype  
---  -
 0   type_of_action                        2500 non-null   int64   
 1   traveled_distance_pixel              2500 non-null   float64  
 2   elapsed_time                         2500 non-null   float64  
 3   direction_of_movement               2500 non-null   int64   
 4   straightness                        2500 non-null   float64  
 5   num_points                          2500 non-null   int64   
 6   sum_of_angles                      2500 non-null   float64  
 7   mean_curv                          2500 non-null   float64  
 8   sd_curv                           2500 non-null   float64  
 9   max_curv                           2500 non-null   float64  
10  min_curv                           2500 non-null   float64  
11  mean_omega                        2500 non-null   float64  
12  sd_omega                         2500 non-null   float64  
13  max_omega                        2500 non-null   float64  
14  min_omega                        2500 non-null   float64  
15  largest_deviation                  2500 non-null   float64  
16  dist_end_to_end_line              2500 non-null   float64  
17  num_critical_points               2500 non-null   int64   
18  mean_vx                          2500 non-null   float64  
19  sd_vx                           2500 non-null   float64  
20  max_vx                          2500 non-null   float64  
21  min_vx                          2500 non-null   float64  
22  mean_vy                          2500 non-null   float64  
23  sd_vy                           2500 non-null   float64  
24  max_vy                          2500 non-null   float64  
25  min_vy                          2500 non-null   float64  
26  mean_v                           2500 non-null   float64  
27  sd_v                            2500 non-null   float64  
28  max_v                            2500 non-null   float64  
29  min_v                            2500 non-null   float64  
30  mean_a                          2500 non-null   float64  
31  sd_a                           2500 non-null   float64  
32  max_a                          2500 non-null   float64  
33  min_a                          2500 non-null   float64  
34  mean_jerk                       2500 non-null   float64  
35  sd_jerk                        2500 non-null   float64  
36  max_jerk                       2500 non-null   float64  
37  min_jerk                       2500 non-null   float64  
38  a_beg_time                     2500 non-null   float64  
39  userid                        2500 non-null   int64   
dtypes: float64(35), int64(5)
memory usage: 781.4 KB
```

Figure 2: "Summary statistics of training data"

Step 3: Check User Distribution

Unique user IDs and counts per user were verified to ensure balanced data. Each user has 250 records in the training dataset and 50 in the testing dataset.

```
[31] train_data['userid'].unique() #display the unique user identification (labels) in the training dataset, accomplished using .unique()
array([12, 15, 16, 20, 21, 23, 29, 35,  7,  9])

[32] test_data['userid'].unique() #display the unique user identification (labels) in the testing dataset, accomplished using .unique()
array([12, 15, 16, 20, 21, 23, 29, 35,  7,  9])
```

Figure 3: "User sample count distribution per ID"

Step 4: Extract Features from Training Dataset

The feature values were separated from the class labels as shown below:

```
X_train = train_data.drop('userid', axis=1)
```

```
Y_train = train_data['userid']
```

After dropping the labels, X_train had shape (2500, 39) and Y_train had shape (2500,).

```
[36] X_train.shape #printing the shape of X_train (training features)
(2500, 39)

[37] Y_train.shape #printing the shape of Y_train (training labels)
(2500,)
```

Figure 4: "Verification of X_train and Y_train shapes"

Step 5: Extract Features from Testing Dataset

Similar to the training dataset, we split the testing dataset:

```
X_test = test_data.drop('userid', axis=1)
```

```
Y_test = test_data['userid']
```

X_test has shape (500, 39) and Y_test has shape (500,)

```
[40] X_test.shape #printing the shape of X_test (test features)
(500, 39)

[42] Y_test.shape #printing the shape of Y_test (test labels)
(500,)
```

```
[43] X_test #printing X_test
```

	type_of_action	traveled_distance_pixel	elapsed_time	direction_of_movement	straightness	num_points	sum_of_angles	mean_curv	sd_curv	max_curv	...	min_v
0	1	2598.696363	9.329	3	0.197868	183	98.685032	0.038825	0.648442	3.141593	...	0.148943
1	4	179.730281	1.919	2	0.957135	36	35.966478	-0.094383	0.735282	1.570796	...	0.000000
2	1	889.719078	2.137	7	0.977541	44	-27.053347	-0.047811	0.413703	1.570796	...	4.255319
3	4	75.236068	1.358	7	0.996951	27	-0.463648	-0.004800	0.081611	0.154549	...	0.000000
4	3	1079.425027	3.541	4	0.948930	66	-31.396924	0.356683	1.644038	5.819538	...	0.000000
...	...	...	...	...	...	...	...	...	...	...	...	...
495	4	5.236068	0.125	1	0.810272	6	2.677945	-0.153004	1.401430	1.570796	...	0.000000
496	1	64.691912	0.874	6	0.893086	57	-65.509798	-0.098650	0.736490	1.570796	...	12.820513
497	1	359.849662	1.310	0	0.811612	121	71.073977	0.098421	1.076820	5.497787	...	15.206597
498	4	67.217478	0.702	7	0.972602	42	-4.248741	0.025096	0.333829	0.785398	...	0.000000
499	1	562.556778	2.871	5	0.622784	116	-20.437963	-0.004052	0.636538	1.666081	...	15.044825

500 rows x 39 columns

```
✓ [44] Y_test #printing Y labels
OK.

userid
0      12
1      12
2      12
3      12
4      12
...     ...
495     9
496     9
497     9
498     9
499     9
500 rows x 1 columns

dtype: int64
```

Figure 5: "Verification of X_{test} and Y_{test} shapes"

Step 6: Train Random Forest Classifier

We trained a Random Forest model with the following configuration:

```
model = RandomForestClassifier(n_estimators=200, random_state=None)
model.fit(X_train, Y_train)
```

```
✓ [45] model = RandomForestClassifier(n_estimators=200, random_state=None) #initialize the random forest classifier
OK.    model.fit(X_train, Y_train) #train the model using .fit() and training features and labels
```

RandomForestClassifier

RandomForestClassifier(n_estimators=200)

Figure 6: "Model Configuration and training output."

Step 7: Evaluate the Model with Cross-Validation

To evaluate the model's performance, 10-fold cross-validation was used:

```
from sklearn.model_selection import cross_validate  
scores = cross_validate(model, X_train, Y_train, cv=10, return_train_score=False)
```

This provided metrics like fit time, score time, and test accuracy for each fold.

```
scores = cross_validate(model, X_train, Y_train, cv=10, return_train_score=False) #calculate the cross validation score and store in a variable score  
print(scores) #print the achieved score  
{'fit_time': array([4.0353601, 3.67203307, 3.2257061, 3.21839786, 4.03088593,  
3.78178978, 3.21517205, 3.45793509, 4.72632718, 3.25692177]), 'score_time': array([0.03964543, 0.04842186, 0.03131986, 0.0268023, 0.05744314,  
0.04250479, 0.02746582, 0.1033814, 0.05083776, 0.02889657]), 'test_score': array([0.384, 0.464, 0.448, 0.516, 0.432, 0.456, 0.444, 0.448, 0.448,  
0.48 ])}
```

Figure 7: Calculate the cross validation score and store in a variable score"

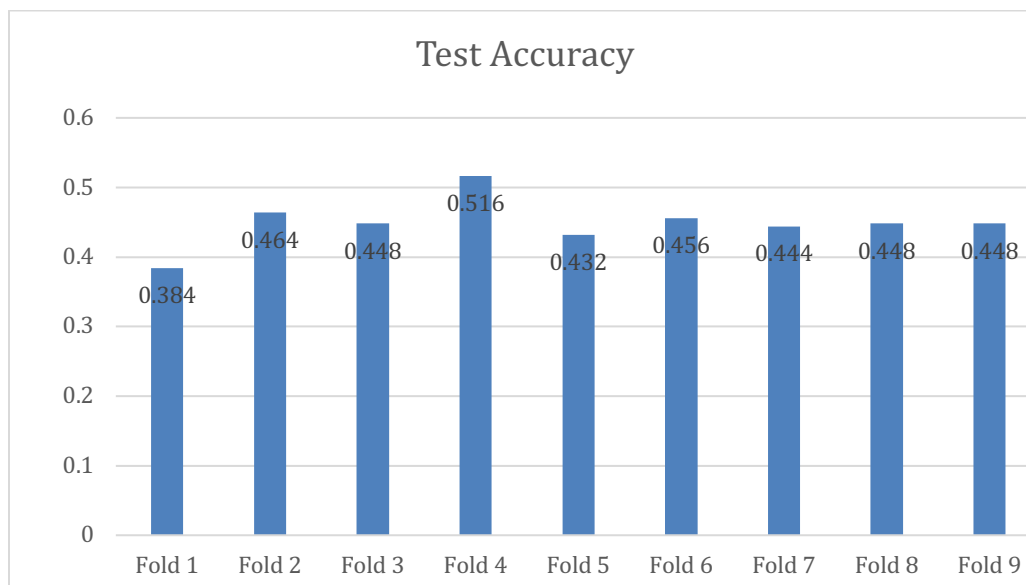


Figure 8: Cross-validation test accuracy per fold (Random Forest, 10-fold CV)

```
#import additional libraries for plotting the confusion matrix and classification report
from sklearn.metrics import confusion_matrix, classification_report
y_pred = model.predict(X_test) #get the predictions using testing features and save them in y_pred
print(y_pred) #print the predictions
print(confusion_matrix(Y_test, y_pred)) #print the confusion matrix
print(classification_report(Y_test, y_pred)) #print the classification report
```

	precision	recall	f1-score	support
7	1.00	1.00	1.00	50
9	1.00	1.00	1.00	50
12	1.00	1.00	1.00	50
15	1.00	1.00	1.00	50
16	1.00	1.00	1.00	50
20	1.00	1.00	1.00	50
21	1.00	1.00	1.00	50
23	1.00	1.00	1.00	50
29	1.00	1.00	1.00	50
35	1.00	1.00	1.00	50
accuracy			1.00	500
macro avg	1.00	1.00	1.00	500
weighted avg	1.00	1.00	1.00	500

Figure 9: "Confusion matrix and classification report – Random Forest classifier accuracy on test set (100%)."

Conclusion

The Random Forest classifier was successfully trained and evaluated using mouse movement data. The steps followed ensured proper separation of features and labels, correct shape validation, and appropriate model training using cross-validation.